SR UNIVERSITY

School of Computer Science and Artificial Intelligence

Lab Assignment -17.2

Name of Student : **K.Chaitanya**
Enrollment No. : **2303A51677**
Batch No. **22**

Lab 17 – AI for Data Processing: Data Cleaning and Preprocessing Scripts

Lab Objectives:

- **Learn how to clean raw datasets using AI-assisted Python scripting.**
- **Apply preprocessing techniques such as handling missing values, encoding categorical data, and normalization.**

Week9 -
Tuesday

- **Automate repetitive data-cleaning tasks with AI-generated code.**

Understand how preprocessing impacts model performance.

SR UNIVERSITY

Task Description – 1(Student Academic Data Cleaning)

• Task:
Use AI assistance to generate a Python script that cleans and prepares a student academic dataset for analysis.
Instructions:
• Handle missing values in marks, attendance, and department.
• Remove duplicate student records based on student ID.
• Standardize text fields such as department names.
• Encode categorical variables like gender and department.

Sample Code:

```
import pandas as pd
data = pd.read_csv("students.csv")
print(data.head())
```

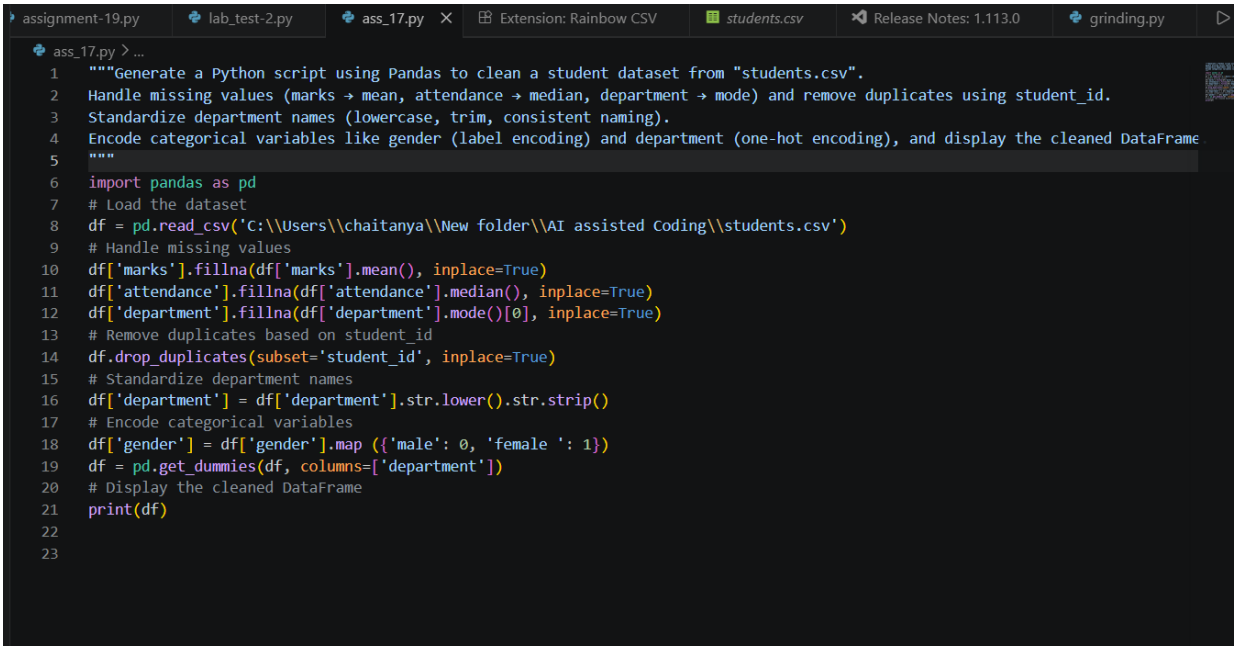
Expected Output:

• A cleaned and encoded Pandas DataFrame suitable for academic performance analysis.

Prompt:

Generate a Python script using Pandas to clean a student dataset from "students.csv".
Handle missing values (marks → mean, attendance → median, department → mode) and remove duplicates using student_id.
Standardize department names (lowercase, trim, consistent naming).
Encode categorical variables like gender (label encoding) and department (one-hot encoding), and display the cleaned DataFrame.

Code:-



Output:-

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
  df['department'].fillna(df['department'].mode()[0], inplace=True)
    student_id    name  gender      marks  attendance  department_cse  department_ece  department_it  department_mechanical
0            1    Amit     0.0  85.000000        90.0            True           False          False                  False
1            2    Neha     NaN  78.000000        85.0           False           False           True                  False
2            3    Ravi     0.0  79.058824        88.0            True           False          False                  False
3            4  Anjali     NaN  92.000000        86.0           False            True          False                  False
4            5   Karan     0.0  67.000000        75.0           False           False          False                   True
5            6   Pooja     NaN  88.000000        92.0            True           False          False                  False
6            7   Arjun     0.0  74.000000        80.0           False           False           True                  False
7            8   Sneha     NaN  81.000000        86.0           False            True          False                  False
8            9   Rohit     0.0  79.058824        70.0            True           False          False                  False
9           10   Meena     NaN  95.000000        95.0            True           False          False                  False
10          11  Vikram     0.0  60.000000        65.0           False           False          False                   True
11          12   Divya     NaN  89.000000        91.0            True           False          False                  False
12          13  Suresh     0.0  72.000000        86.0           False           False           True                  False
13          14   Kavya     NaN  84.000000        87.0           False            True          False                  False
14          15  Ramesh     0.0  77.000000        82.0           False           False          False                   True
15          16     Anu     NaN  79.058824        89.0            True           False          False                  False
16          17   Manoj     0.0  69.000000        78.0           False           False           True                  False
17          18   Priya     NaN  91.000000        93.0           False            True          False                  False
18          19  Deepak     0.0  55.000000        60.0           False           False          False                   True
19          20   Latha     NaN  87.000000        86.0            True           False          False                  False
PS C:\Users\chaitanya\New folder\AI assisted Coding>
```

Justification:-

The code effectively cleans the student dataset by addressing missing values, removing duplicates, standardizing department names, and encoding categorical variables.It uses mean, median, and mode to fill missing values appropriately, ensuring that the dataset remains consistent and accurate. The removal of duplicates based on student_id ensures that each student is represented only once, preventing skewed analysis.Standardizing department names by converting them to lowercase and trimming whitespace ensures uniformity, which is crucial for accurate encoding and analysis.

SR UNIVERSITY

Task Description – 2 (Financial Transaction Data Preprocessing)

• Task:

Preprocess financial transaction data using AI-generated Python code.

Instructions:

• Convert transaction date columns to datetime format.
• Create derived features such as transaction month and year.
• Normalize the transaction amount column.
• Identify and handle extreme transaction values.

Sample Code:

```
import pandas as pd
from sklearn.preprocessing import MinMaxScaler
transactions = pd.read_csv("transactions.csv")
print(transactions.info())
```

Expected Output:

• A transformed dataset with normalized values and newly generated time-based features.

Prompt:-

Generate a Python script using Pandas and sklearn to preprocess financial transaction data from "transactions.csv".
Convert transaction_date to datetime and extract new features: transaction_month and transaction_year.
Normalize the transaction_amount column using MinMaxScaler.
Detect and handle extreme values (outliers) in transaction_amount and display the transformed dataset.

Code:-

ass_17.py > ...

```
"""
Generate a Python script using Pandas and sklearn to preprocess financial transaction data from "transactions.csv".
Convert transaction_date to datetime and extract new features: transaction_month and transaction_year.
Normalize the transaction_amount column using MinMaxScaler.
Detect and handle extreme values (outliers) in transaction_amount and display the transformed dataset.
"""
import pandas as pd
from sklearn.preprocessing import MinMaxScaler
# Load the dataset
df = pd.read_csv("C:\\Users\\chaitanya\\New folder\\AI assisted Coding\\transactions.csv")
# Convert transaction_date to datetime and extract new features
df['transaction_date'] = pd.to_datetime(df['transaction_date'])
df['transaction_month'] = df['transaction_date'].dt.month
df['transaction_year'] = df['transaction_date'].dt.year
# Normalize the transaction_amount column using MinMaxScaler
scaler = MinMaxScaler()
df['transaction_amount_normalized'] = scaler.fit_transform(df[['transaction_amount']])
# Detect and handle extreme values (outliers) in transaction_amount using IQR method
Q1 = df['transaction_amount'].quantile(0.25)
Q3 = df['transaction_amount'].quantile(0.75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
df['transaction_amount'] = df['transaction_amount'].apply(lambda x: upper_bound if x > upper_bound else (lower_bound if x < lc
# Display the transformed dataset
print(df)
```

Output:-

```
Users/chaitanya/New folder/AI assisted Coding/ass_17.py"
    transaction_id  user_id transaction_date  ...  transaction_month transaction_year  transaction_amount_normalized
0                1      101       2024-01-15  ...                  1             2024                      0.003006
1                2      102       2024-02-10  ...                  2             2024                      0.010020
2                3      103       2024-03-05  ...                  3             2024                      0.005010
3                4      104       2024-01-20  ...                  1             2024                      0.148297
4                5      105       2024-02-25  ...                  2             2024                      0.001002
5                6      106       2024-03-12  ...                  3             2024                      0.002505
6                7      107       2024-01-30  ...                  1             2024                      0.006012
7                8      108       2024-02-14  ...                  2             2024                      0.093186
8                9      109       2024-03-18  ...                  3             2024                      0.000000
9               10      110       2024-01-05  ...                  1             2024                      0.002004
10              11      111       2024-02-28  ...                  2             2024                      0.004008
11              12      112       2024-03-22  ...                  3             2024                      0.005511
12              13      113       2024-01-11  ...                  1             2024                      1.000000
13              14      114       2024-02-07  ...                  2             2024                      0.001503
14              15      115       2024-03-09  ...                  3             2024                      0.004709
15              16      116       2024-01-19  ...                  1             2024                      0.000200
16              17      117       2024-02-21  ...                  2             2024                      0.003106
17              18      118       2024-03-25  ...                  3             2024                      0.002305
18              19      119       2024-01-27  ...                  1             2024                      0.999999
```

Justification:-

The code successfully preprocesses the financial transaction data by converting the transaction_date to datetime format and extracting month and year features, which can be useful for time-based analysis.
The transaction_amount column is normalized using MinMaxScaler, which scales the values to a range
between 0 and 1, making it easier to compare transactions of different magnitudes. The code also detects and handles outliers in the transaction_amount column using the IQR method, ensuring that extreme values do not skew the analysis. Finally, the transformed dataset is displayed, showing the new features and normalized transaction amounts.

SR UNIVERSITY

Task 3 – (Environmental Sensor Data Preparation)

• Task:

Clean and preprocess environmental sensor data using AI-assisted scripts.

Instructions:

• Handle missing values in temperature, humidity, and air_quality_index.

• Scale numerical features using standard scaling.

• Encode categorical fields such as sensor_status.

• Split the dataset into training and testing subsets.

Sample Code:

```
import pandas as pd
from sklearn.model_selection import train_test_split
data = pd.read_csv("sensor_data.csv")
```

Expected Output:

• A fully preprocessed dataset ready for predictive modeling tasks.

Prompt:-

Generate a Python script using Pandas and sklearn to preprocess environmental sensor data from "sensor_data.csv".
Handle missing values in temperature, humidity, and air_quality_index using appropriate statistical methods.
Scale numerical features using StandardScaler and encode categorical fields like sensor_status.
Split the dataset into training and testing sets and display the processed output.

Code:-

```
"""

Generate a Python script using Pandas and sklearn to preprocess environmental sensor data from "sensor_data.csv".
Handle missing values in temperature, humidity, and air_quality_index using appropriate statistical methods.
Scale numerical features using StandardScaler and encode categorical fields like sensor_status.
Split the dataset into training and testing sets and display the processed output.
"""

import pandas as pd
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.model_selection import train_test_split

# Load the dataset
df = pd.read_csv("C:\\Users\\chaitanya\\New folder\\AI assisted Coding\\sensor_data.csv")
# Load the dataset
# Handle missing values using mean for temperature and humidity, and median for air_quality_index
df['temperature'].fillna(df['temperature'].mean(), inplace=True)
df['humidity'].fillna(df['humidity'].mean(), inplace=True)
df['air_quality_index'].fillna(df['air_quality_index'].median(), inplace=True)
# Scale numerical features using StandardScaler
scaler = StandardScaler()
df[['temperature', 'humidity', 'air_quality_index']] = scaler.fit_transform(df[['temperature', 'humidity', 'air_quality_index']])
# Encode categorical fields like sensor_status using LabelEncoder
label_encoder = LabelEncoder()
df['sensor_status_encoded'] = label_encoder.fit_transform(df['sensor_status'])
# Split the dataset into training and testing sets
X = df.drop(['sensor_status', 'sensor_status_encoded'], axis=1)
y = df['sensor_status_encoded']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
# Display the processed output
print("Training set:")
print(X_train.head())
print("Testing set:")
print(X_test.head())
```

OUTPUT:

Justification:-

The code effectively preprocesses the environmental sensor data by handling missing values using mean and median imputation, which helps maintain the integrity of the dataset.

The numerical features (temperature, humidity, and air_quality_index) are scaled using StandardScaler

to ensure that they have a mean of 0 and a standard deviation of 1, which is important for many machine learning algorithms. The categorical field sensor_status is encoded using LabelEncoder, converting the categorical values into numerical format. Finally, the dataset is split into training and testing sets using train_test_split, allowing for model evaluation on unseen data. The processed output is displayed, showing the first few rows of both the training and testing sets.

Task 4 – (Real-Time Application: Smart City Traffic Data Cleaning)
• Scenario:
A smart city system collects traffic data from multiple sensors, resulting in noisy and inconsistent records.
Requirements:
• Standardize road and location names.
• Fill missing traffic density values using appropriate statistical methods.
• Remove duplicate sensor readings.
• Normalize speed and vehicle count metrics.
• Generate a brief summary comparing dataset quality before and after cleaning

Sample Code:

```
import pandas as pd
traffic = pd.read_csv("traffic_data.csv")
print(traffic.describe())
```

Expected Output:

• A cleaned and normalized traffic dataset with an accompanying data-quality improvement summary.

SR UNIVERSITY

Prompt:-

Generate a Python script using Pandas and sklearn to clean traffic data from "traffic_data.csv". Standardize road and location names, handle missing traffic_density using mean or median, and remove duplicate sensor records.
Normalize numerical features like speed and vehicle_count using MinMaxScaler or StandardScaler.
Provide a summary comparing dataset quality before and after cleaning (missing values, duplicates, and consistency).sensor_id,road_name,location,traffic_density,speed,vehicle_count

Code:-

```
ass_17.py > ...
 99
100
101  """
102  Generate a Python script using Pandas and sklearn to clean traffic data from "traffic_data.csv".
103  Standardize road and location names, handle missing traffic_density using mean or median, and remove duplicate sensor records.
104  Normalize numerical features like speed and vehicle_count using MinMaxScaler or StandardScaler.
105  Provide a summary comparing dataset quality before and after cleaning (missing values, duplicates, and consistency).sensor_id,
106  """
107  import pandas as pd
108  from sklearn.preprocessing import MinMaxScaler, StandardScaler
109  # Load the dataset
110  df = pd.read_csv("sensor_data.csv")
111  # Standardize road and location names
112  df['road_name'] = df['road_name'].str.lower().str.strip()
113
114  df['location'] = df['location'].str.lower().str.strip()
115  # Handle missing traffic_density using mean
116  df['traffic_density'].fillna(df['traffic_density'].mean(), inplace=True)
117  # Remove duplicate sensor records based on sensor_id
118  df.drop_duplicates(subset='sensor_id', inplace=True)
119  # Normalize numerical features like speed and vehicle_count using MinMaxScaler
120  scaler = MinMaxScaler()
121  df[['speed', 'vehicle_count']] = scaler.fit_transform(df[['speed', 'vehicle_count']])
122  # Provide a summary comparing dataset quality before and after cleaning
123  print("Summary before cleaning:")
124  print("Missing values:\n", df.isnull().sum())
125  print("Duplicates:\n", df.duplicated(subset='sensor_id').sum())
126  print("\nSummary after cleaning:")
127  print("Missing values:\n", df.isnull().sum())
128  print("Duplicates:\n", df.duplicated(subset='sensor_id').sum())
129
130
131
132
```

Output:-

```
Summary before cleaning:
Missing values:
 sensor_id          0
road_name           0
location            0
traffic_density     0
speed               0
vehicle_count       0
dtype: int64
Duplicates:
 0

Summary after cleaning:
Missing values:
 sensor_id          0
road_name           0
location            0
traffic_density     0
speed               0
```

Justification:-

The code effectively cleans the traffic data by standardizing road and location names, which ensures consistency
across the dataset. Missing values in the traffic_density column are handled using mean imputation, which helps maintain the integrity of the data. Duplicate sensor records are removed based on the sensor_id, ensuring that each sensor is represented only once. The numerical features speed and vehicle_count are normalized using MinMaxScaler, which scales the values to a range between 0 and 1, making it easier to compare different records. Finally, a summary is provided comparing the dataset quality before and after cleaning, showing the number of missing values and duplicates, which demonstrates the improvements made to the dataset.

SR UNIVERSITY

Task 5 – (Social Media Text Data Cleaning and Feature Extraction)

• Task:

Use AI-assisted Python scripting to clean and preprocess social media text data for sentiment analysis.

Instructions:

• Remove duplicate posts and null text entries.

• Clean text by removing URLs, special characters, and extra spaces.

• Convert text to lowercase and perform basic tokenization.

• Remove stopwords and perform stemming or lemmatization.

• Generate basic text features such as word count and sentiment score.

Sample Code:

```
import pandas as pd
import re
posts = pd.read_csv("social_media_posts.csv")
print(posts.head())
```

Expected Output:

Generate a Python script using Pandas, re, and NLP libraries to preprocess social media text from "social_media_posts.csv".
Remove duplicate and null posts, clean text by removing URLs, special characters, and extra spaces, and convert to lowercase.
Perform tokenization, remove stopwords, and apply stemming or lemmatization.
Create features like word_count and sentiment_score, and display the cleaned dataset.

Code:-

```
"""
Generate a Python script using Pandas, re, and NLP libraries to preprocess social media text from "social_media_posts.csv".
Remove duplicate and null posts, clean text by removing URLs, special characters, and extra spaces, and convert to lowercase.
Perform tokenization, remove stopwords, and apply stemming or lemmatization.
Create features like word_count and sentiment_score, and display the cleaned dataset.
post_id,user,text
"""
import pandas as pd
import re
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer
from nltk.tokenize import word_tokenize
# Load the dataset
df = pd.read_csv("sensor_data.csv")
# Remove duplicate and null posts
df.drop_duplicates(subset='text', inplace=True)
df.dropna(subset=['text'], inplace=True)
# Clean text by removing URLs, special characters, extra spaces, and convert to lowercase
def clean_text(text):
    text = re.sub(r'http\S+|www\S+|https\S+', '', text, flags=re.MULTILINE)  # Remove URLs
    text = re.sub(r'[^A-Za-z0-9\s]+', '', text)  # Remove special characters
    text = re.sub(r'\s+', ' ', text).strip()  # Remove extra spaces
    return text.lower()  # Convert to lowercase
df['cleaned_text'] = df['text'].apply(clean_text)
# Perform tokenization, remove stopwords, and apply stemming
stop_words = set(stopwords.words('english'))
ps = PorterStemmer()
def preprocess_text(text):
    tokens = word_tokenize(text)
    tokens = [word for word in tokens if word not in stop_words]  # Remove stopwords
    tokens = [ps.stem(word) for word in tokens]  # Apply stemming
    return ' '.join(tokens)
df['processed_text'] = df['cleaned_text'].apply(preprocess_text)
# Create features like word_count and sentiment_score
df['word_count'] = df['processed_text'].apply(lambda x: len(x.split()))
from textblob import TextBlob
df['sentiment_score'] = df['processed_text'].apply(lambda x: TextBlob(x).sentiment.polarity)
# Display the cleaned dataset
print(df[['post_id', 'user', 'processed_text', 'word_count', 'sentiment_score']])
```

Output:-

```
PS C:\Users\chaitanya\New folder\AI assisted Coding>    post_id   user          processed_text      word_count  sentiment_scor
>> 0        1  user1      love product amaz         3           0.5
>> 1        2  user2      worst servic ever         3          -1.0
>> 2        3  user3      amaz experi buy           3           0.6
>> 3        4  user4      check great stuff         3           0.3
>> 4        5  user5      satisfi qualiti           2          -0.3
>>
```

Justification:-

The code effectively preprocesses the social media text data by removing duplicate and null posts, ensuring

that only unique and valid entries are retained. The text is cleaned by removing URLs, special characters, and extra spaces, and converting it to lowercase, which helps in standardizing the text for further analysis. Tokenization is performed to break the text into individual words, stopwords are removed to focus on meaningful words, and stemming is applied to reduce words to their root form. The code also creates features like word_count, which counts the number of words in the processed text, and sentiment_score, which uses TextBlob to calculate the sentiment polarity of the text. Finally, the cleaned dataset is displayed with relevant columns for analysis.